

LINQ ToDictionary Method in C# with Examples

In this article, I will discuss the **LINQ ToDictionary Method in C#** with Examples. Please read our previous article discussing the [LINQ ToList and ToArray Methods in C#](#) with Examples. Like the LINQ ToList and ToArray, the ToDictionary method also belongs to the conversion operator category.

LINQ ToDictionary Method in C#:

In C#, LINQ (Language Integrated Query) provides a set of methods for querying objects that implement `IEnumerable<T>`. One such method is `ToDictionary`, which converts a collection into a `Dictionary<TKey, TValue>` based on a key selector function and an optional element selector function.

The `ToDictionary` method in C# is a LINQ extension method that converts a sequence of elements (e.g., a collection or query result) into a dictionary. It is particularly useful when you have a collection of objects with key-value pairs, and we want to create a dictionary where the keys are derived from the elements in the collection.

The `ToDictionary` method is defined in the `System.Linq` namespace can create a dictionary from an enumerable collection where each element is transformed into a key-value pair. This method causes the query to be executed immediately. There are four overloaded versions available for this method. Let us start the discussion with the following overloaded version.

```
public static Dictionary<TKey, TSource> ToDictionary<TSource, TKey>(this  
IEnumerable<TSource> source, Func<TSource, TKey> keySelector);
```

Parameters:

This method takes two parameters. They are as follows:

1. **source**: It is the **Collections.Generic.IEnumerable<T>** collection from which we need to create a **System.Collections.Generic.Dictionary<TKey, TValue>** collection.
2. **keySelector**: It is a function that is basically used to extract a key from each element.

Type Parameters:

1. **TSource**: The type of elements of the source sequence.
2. **TKey**: The type of the key returned by the key Selector.

Returns:

1. It returns a **System.Collections.Generic.Dictionary<TKey, TValue>** collection that contains keys and values.

Exceptions:

This method throws the following two exceptions.

1. It throws **ArgumentNullException** when the source or **keySelector** is **null** or the **keySelector** function produces a null key.
2. Throws **ArgumentException** when the **keySelector** produces duplicate keys for two elements.

Example to Convert a List to a Dictionary in C#.

Here, in the following example, the product ID is the key, and the Product is its value.

```
using System.Linq;
using System.Collections.Generic;
using System;
namespace ConversionOperators
{
    public class Product
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
    class Program
    {
        public static void Main()
        {
            List<Product> listProducts = new List<Product>
            {
                new Product { ID= 1001, Name = "Mobile", Price = 800 },
                new Product { ID= 1002, Name = "Laptop", Price = 900 },
                new Product { ID= 1003, Name = "Desktop", Price = 800 }
            };
            Dictionary<int, Product> productsDictionary = listProducts.ToDictionary(
                x.ID);
            foreach (KeyValuePair<int, Product> kvp in productsDictionary)
            {
                Console.WriteLine(kvp.Key + " Name : " + kvp.Value.Name + ", Price : " +
                kvp.Value.Price);
            }
            Console.ReadKey();
        }
    }
}
```

When you run the above application, it will give the output as expected, as shown below.

```
1001 Name : Mobile, Price: 800
1002 Name : Laptop, Price: 900
1003 Name : Desktop, Price: 800
```

Note: Keep in mind that if there are duplicate keys in the source collection, the ToDictionary method will throw an exception. You can handle this situation by providing a custom key selector or using the ToLookup

method if duplicate keys are expected.

Another Overloaded Version of the ToDictionary Method in C#:

The following **ToDictionary** method Creates a **System.Collections.Generic.Dictionary<TKey, TValue>** collection from the **System.Collections.Generic.IEnumerable<T>** according to the specified **key selector** and **element selector**.

```
public static Dictionary<TKey, TElement> ToDictionary<TSource, TKey, TElement>(this
IEnumerable<TSource> source, Func<TSource, TKey> keySelector, Func<TSource,
TElement> elementSelector);
```

Parameters:

1. **source**: It is the source **System.Collections.Generic.IEnumerable<T>** collection from where we need to create a **System.Collections.Generic.Dictionary<TKey, TValue>**.
2. **keySelector**: A function to extract a key from each element.
3. **elementSelector**: A transform function to produce a result element value from each element.

Type Parameters:

1. **TSource**: The type of elements of source.
2. **TKey**: The type of the key returned by the key Selector.
3. **TElement**: The type of value returned by the element selector.

Returns:

1. It returns a **System.Collections.Generic.Dictionary<TKey, TValue>** that contains values of type **TElement** selected from the input sequence.

Exceptions:

1. It throws **System.ArgumentNullException** when the **source** or **key selector** is null or the **key selector** function produces a null key.
2. It also throws **System.ArgumentException** when the **key selector** produces duplicate keys for two elements.

Example to Understand the above LINQ ToDictionary Method in C#:

In the following example, we convert **List<Product>** to a **Dictionary**. Here, the product ID is the key, and the Product name is its value.

```
using System.Linq;
using System.Collections.Generic;
using System;
namespace ConversionOperators
```

```

{
    public class Product
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
    class Program
    {
        public static void Main()
        {
            List<Product> listProducts = new List<Product>
            {
                new Product { ID= 1001, Name = "Mobile", Price = 800 },
                new Product { ID= 1002, Name = "Laptop", Price = 900 },
                new Product { ID= 1003, Name = "Desktop", Price = 800 }
            };
            Dictionary<int, string> productsDictionary = listProducts.ToDictionary(
x.ID, x => x.Name);
            foreach (KeyValuePair<int, string> kvp in productsDictionary)
            {
                Console.WriteLine("Key : " + kvp.Key + " Value : " + kvp.Value);
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

Key : 1001 Value : Mobile
Key : 1002 Value : Laptop
Key : 1003 Value : Desktop

```

What happens when the key is the same for two elements?

In the following example, it throws a **System.ArgumentException** as there are two products with the same ID (i.e., Products with id 1001), and we are using ID as the key for the dictionary.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace ConversionOperators
{
    public class Product
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
}

```

```

    }
    class Program
    {
        public static void Main()
        {
            List<Product> listProducts = new List<Product>
            {
                new Product { ID= 1001, Name = "Mobile", Price = 800 },
                new Product { ID= 1001, Name = "Laptop", Price = 900 },
                new Product { ID= 1003, Name = "Desktop", Price = 800 }
            };
            Dictionary<int, string> productsDictionary = listProducts.ToDictionary(
x.ID, x => x.Name);
            foreach (KeyValuePair<int, string> kvp in productsDictionary)
            {
                Console.WriteLine("Key : " + kvp.Key + " Value : " + kvp.Value);
            }
            Console.ReadKey();
        }
    }
}

```

Note: Remember that the keySelector must select unique keys, as dictionaries do not allow duplicate keys. If there are duplicates, ToDictionary will throw an ArgumentException. To handle duplicates, you could use a method like GroupBy before calling ToDictionary or handle the duplicates in a way that fits your specific requirements.

What happens when the source is null?

In the following example, it will throw **System.ArgumentNullException** as the source (i.e., listProducts) is null.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace ConversionOperators
{
    public class Product
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
    class Program
    {
        public static void Main()
        {
            List<Product> listProducts = null;
            Dictionary<int, string> productsDictionary = listProducts.ToDictionary(
x.ID, x => x.Name);

```

```

        foreach (KeyValuePair<int, string> kvp in productsDictionary)
        {
            Console.WriteLine("Key : " + kvp.Key + " Value : " + kvp.Value);
        }
        Console.ReadKey();
    }
}

```

When to use the LINQ ToDictionary Method in C#?

You should use the ToDictionary method in C# when you have a collection of data, and you want to transform that data into a dictionary where you can easily look up values by their associated keys. Here are some scenarios when you might consider using the ToDictionary method:

Converting a Collection to a Dictionary:

When you have a collection of objects or elements, you need to organize them into a dictionary for efficient key-based access.

```
var dictionary = collection.ToDictionary(keySelector);
```

Projecting Data:

When you want to transform a collection by projecting certain properties or values into keys and values in a dictionary.

```
var dictionary = collection.ToDictionary(keySelector, valueSelector);
```

Grouping Data:

When you want to group elements from a collection into a dictionary based on a common property or key.

```
var groupedDictionary = collection.GroupBy(keySelector).ToDictionary(group => group.Key, group => group.ToList());
```

Indexing Data for Quick Lookups:

When you have a collection of items and want to build an index to retrieve items by specific property or key efficiently.

```
var itemIndex = collection.ToDictionary(item => item.KeyProperty);
```

Creating Dictionaries From Query Results:

When working with LINQ queries, you may want to convert query results into dictionaries for further processing or data manipulation.

```
var result = dbContext.Customers
    .Where(customer => customer.IsActive)
    .ToDictionary(customer => customer.Id);
```

Merging or Aggregating Data:

When you want to merge data from multiple sources into a single dictionary, combine values based on a common key.

```
var dictionary1 = source1.ToDictionary(keySelector);  
var dictionary2 = source2.ToDictionary(keySelector);  
var mergedDictionary = dictionary1.Concat(dictionary2).ToDictionary(pair => pair.Key, pair => pair.Value);
```

When Not to Use LINQ ToDictionary Method in C#?

While the ToDictionary method in C# is a useful tool for converting collections or query results into dictionaries, there are situations where it may not be the best choice. Here are some scenarios in which you might want to avoid using ToDictionary:

- **Duplicate Keys:** If the source collection or query result contains duplicate keys, using ToDictionary will throw an exception. In such cases, consider using the ToLookup method if you need to handle multiple values associated with the same key.
- **Modifying Existing Dictionary:** If you have an existing dictionary that you want to modify, it's often more efficient to directly add or update entries using the dictionary's methods (Add, Remove, etc.) rather than creating a new dictionary with ToDictionary. Creating a new dictionary can be less efficient and consumes more memory.
- **Expensive Key or Value Transformations:** If the key or value transformation logic provided to ToDictionary is computationally expensive or involves complex operations, repeatedly using ToDictionary in performance-critical scenarios may result in unnecessary overhead. In such cases, consider precomputing the dictionary or optimizing your code.
- **Deferred Execution Concerns:** Be cautious when using ToDictionary with LINQ queries with deferred execution. The dictionary will be populated when the query is executed, not when ToDictionary is called. If the underlying data source changes between the time you call ToDictionary and the execution of the query, you may get unexpected results.
- **Memory Usage:** Creating a dictionary consumes memory, and if you have a large collection, converting it to a dictionary can lead to increased memory usage. Be mindful of memory constraints, especially in scenarios where memory is limited.
- **Performance Considerations:** In some cases, using a foreach loop or other techniques may be more performant than using LINQ methods like ToDictionary, especially for simple data transformations or when working with large datasets.
- **Simplicity:** If you don't need the key-value structure provided by a dictionary and are just iterating over the data or performing other operations, using ToDictionary might introduce unnecessary complexity to your code.

So, the ToDictionary method is useful whenever you need to convert data from a collection or query result into a key-value data structure for efficient lookups, grouping, or indexing purposes. It's a powerful tool in LINQ that can simplify many data manipulation tasks in C#.

In the next article, I will discuss the need and use of the [LINQ Cast Method in C#](#) with Examples. Here, in this article, I try to explain the need and use of the **LINQ ToDictionary Method in C#** with Examples. I hope you enjoy this article.